СЕРВЕР API «МАРШРУТА»: ЗАПУСК И ПОДКЛЮЧЕНИЕ ТЕЛЕФОНА

Приложение «Маршрут» не хранит данные у себя. Все пользователи, планы, занятия, заметки, свои темы и избранное лежат в базе данных SQLite на сервере, а приложение получает, добавляет, изменяет и удаляет их только запросами к API сервера. Сервер запускается на ноутбуке с Windows, телефон подключается к нему по Wi-Fi.

Что понадобится:

- ноутбук с Windows 10 или 11;
- телефон (iPhone или Android) с установленным «Маршрутом»;
- одна сеть Wi-Fi, к которой подключены и ноутбук, и телефон.

Шаг 1. Скачать сервер

1. Открыть страницу сборок «Сборка и тесты» (войти в GitHub, если попросит).
2. Нажать на верхний запуск с зелёной галочкой.
3. Прокрутить страницу вниз до раздела **Artifacts** и нажать **marshrut-server-windows**. Скачается архив `marshrut-server-windows.zip`.
4. Открыть папку «Загрузки», нажать на архив правой кнопкой мыши, выбрать **«Извлечь все…»**, затем **«Извлечь»**.

В открывшейся папке лежат файл **«Запустить сервер»** и папки `bin`, `lib`, `catalog`. Папку лучше перенести в удобное место, например на рабочий стол: в ней же будет храниться база данных.

Когда на странице Releases публикуется новый релиз, архив `marshrut-server-windows.zip` прикладывается к нему автоматически, и его можно скачать оттуда.

Шаг 2. Запустить сервер

1. Дважды щёлкнуть файл **«Запустить сервер»**.

2. Если Windows покажет окно **«Windows защитила ваш компьютер»**, нажать **«Подробнее»**, затем **«Выполнить в любом случае»**.
3. Появится окно **«Брандмауэр Защитника Windows заблокировал некоторые функции этого приложения»**. Поставить обе галочки: **«Частные сети»** и **«Общественные сети»**, затем нажать **«Разрешить доступ»**. Без этого телефон не сможет подключиться к серверу.
4. В чёрном окне сервера появятся строки:

```
Сервер «Маршрут» запущен.
Адрес для приложения на этом компьютере:
  http://localhost:8080
Адрес для телефона в той же сети Wi-Fi:
  http://192.168.1.5:8080
```

Цифры во второй строке у вас будут свои. Этот адрес нужен для телефона. Если адресов несколько, обычно подходит тот, что начинается с `192.168.`

Окно сервера не закрывать, пока работаете с приложением. Остановить сервер можно сочетанием клавиш **Ctrl+C** или закрыв окно. Данные при этом не пропадут: они сохранены в файле `marshrut_server.sqlite` в папке сервера.

Шаг 3. Подключить телефон

1. Подключить телефон к той же сети Wi-Fi, что и ноутбук.
2. Проверить связь: открыть на телефоне браузер (Safari или Chrome) и ввести адрес из окна сервера, например `http://192.168.1.5:8080`. Должна открыться страница **«Сервер “Маршрут” работает»**. Если страница не открылась, смотрите раздел «Если не получается».
3. Открыть «Маршрут». На экране входа внизу нажать кнопку **«Сервер: localhost:8080»**.
4. Стереть старый адрес, ввести адрес из окна сервера (например, `192.168.1.5:8080`) и нажать **«Проверить и сохранить»**.
5. На iPhone появится вопрос о доступе к устройствам в локальной сети. Нажать **«Разрешить»**.

6. Внизу экрана появится надпись **«Связь с сервером есть, адрес сохранён»**. Теперь можно зарегистрироваться и войти.

Адрес запоминается: в следующий раз вводить его не нужно, если адрес ноутбука в сети не поменялся. Текущий адрес сервера виден в приложении в разделе **«Профиль»**, строка **«Сервер API»**.

Приложение для Windows на том же ноутбуке подключается к серверу само: ему подходит адрес по умолчанию `http://localhost:8080`, поэтому вводить ничего не нужно.

Как показать работу через API на защите

1. Запустить сервер на ноутбуке и поставить окно сервера так, чтобы его видела комиссия.
2. На телефоне войти в «Маршрут» и выполнить действия: создать заметку, изменить её, удалить, отметить занятие выполненным.
3. После каждого действия в окне сервера появляется строка журнала, например:

```
10:15:02  POST   /api/notes  →  201  (12 мс)
10:15:20  PUT    /api/notes/3  →  204  (8 мс)
10:15:31  DELETE /api/notes/3  →  204  (6 мс)
```

Здесь видно, какой запрос пришёл (POST — добавление, GET — получение, PUT и PATCH — изменение, DELETE — удаление), по какому адресу и с каким кодом ответил сервер.

4. Показать обработку ошибок: остановить сервер (Ctrl+C) и на телефоне отметить занятие или сохранить заметку. Внизу экрана появится красное сообщение **«Нет связи с сервером … Проверьте, что сервер запущен и телефон в той же сети Wi-Fi»**, а данные не изменятся. Снова запустить сервер и повторить действие: теперь оно сохранится, а в окне сервера появится строка журнала.

Подготовка к защите

- Проверить всё заранее дома: запустить сервер, подключить телефон, зарегистрироваться, создать план.

- В сети колледжа устройства иногда не видят друг друга. Тогда включить на телефоне точку доступа (на iPhone: **Настройки → Режим модема → «Разрешать другим»**), подключить к ней ноутбук, перезапустить сервер и ввести в приложении новый адрес из окна сервера.
- База данных остаётся в файле `marshrut_server.sqlite`: созданные дома аккаунт и план будут на месте и на защите, если запускать сервер из той же папки.

Если не получается

- **Страница сервера не открывается в браузере телефона.** Проверить, что телефон и ноутбук в одной сети Wi-Fi и что окно сервера открыто. Если при первом запуске не нажали «Разрешить доступ», открыть **Пуск → «Брандмауэр Защитника Windows» → «Разрешение взаимодействия с приложением или компонентом в брандмауэре Защитника Windows» → «Изменить параметры»**, найти в списке `marshrut_server` и поставить галочки «Частная» и «Публичная», нажать **«ОК»**.
- **На iPhone нет связи, хотя в браузере страница открывается.** Открыть **Настройки → Конфиденциальность и безопасность → Локальная сеть** и включить «Маршрут».
- **Окно сервера пишет «Не удалось занять порт 8080».** Сервер уже запущен в другом окне. Закрыть лишнее окно или работать с уже открытым.
- **Приложение пишет «Сервер … не отвечает».** Адрес ноутбука мог поменяться (например, после переподключения к Wi-Fi). Перезапустить сервер, посмотреть новый адрес и ввести его в приложении.

Как устроена работа с данными

Приложение и сервер — две отдельные программы, они общаются по протоколу HTTP, данные передаются в формате JSON:

- **приложение** (Flutter): экраны берут данные у провайдеров, провайдеры — у репозиториев API, а репозитории отправляют запросы через общий клиент API. Клиент добавляет к запросу токен входа, ждёт ответ не дольше 10 секунд и превращает любые ошибки в понятные сообщения на русском;
- **сервер** (Dart, библиотека shelf): по адресу и методу запроса выбирает обработчик, проверяет токен, проверяет данные теми же правилами, что и формы приложения, проверяет, что запись принадлежит этому пользователю, и только после этого читает или изменяет базу данных SQLite (библиотека drift).

Пароли хранятся на сервере только в виде хэша PBKDF2. После входа сервер выдаёт токен, приложение отправляет его в заголовке `Authorization: Bearer …` с каждым запросом. Токен действует 30 дней, а при выходе из аккаунта удаляется на сервере.

Ответы сервера и ошибки

Таблица 1 — Коды ответов сервера и что видит пользователь

Код	**Когда**	**Что показывает приложение**
200	Данные получены или изменены	Данные на экране
201	Запись создана (ответ содержит её id)	Новая запись в списке
204	Действие выполнено, данных в ответе нет	Обновлённый экран
400	Неверные данные: пустое поле, занятый email, неверный пароль	Текст ошибки под полем или внизу экрана
401	Нет токена, токен устарел или аккаунт удалён	Переход на экран входа
404	Запись не найдена или принадлежит другому пользователю	«План не найден», «Заметка не найдена»
500	Внутренняя ошибка сервера (подробности только в окне сервера)	«Внутренняя ошибка сервера»
—	Сервер выключен, нет сети, неверный адрес	«Нет связи с сервером …»
—	Сервер не ответил за 10 секунд	«Сервер … не отвечает»

Если ошибка случилась при действии (сохранение, удаление, отметка), сообщение появляется внизу экрана красным цветом. Если не загрузился экран, вместо данных показывается сообщение об ошибке, а списки сами пробуют загрузиться снова каждые 5 секунд.

Пример запроса и ответа при создании заметки:

```
POST /api/notes
Authorization: Bearer <токен>
Content-Type: application/json

{"title": "Списки в Python", "body": "append добавляет в конец", "topicId":
"py_lists"}

Ответ 201:
{"id": 3}
```

Пример ошибки:

```
POST /api/notes   (без поля title)

Ответ 400:
{"error": "Поле «title» не заполнено или имеет неверный тип"}
```

Адреса API

Таблица 2 — Адреса API сервера

Метод	**Адрес**	**Что делает**
GET	/api/health	Проверка связи (приложение вызывает при сохранении адреса)
GET	/api/directions	Каталог направлений и тем
POST	/api/auth/register	Регистрация
POST	/api/auth/login	Вход: ответ содержит токен
POST	/api/auth/logout	Выход: токен удаляется
GET	/api/me	Данные своего профиля
PUT	/api/me	Изменение имени, email и цвета аватара
PUT	/api/me/password	Смена пароля
PUT	/api/me/reminder	Время ежедневного напоминания
DELETE	/api/me	Удаление аккаунта со всеми данными
GET	/api/plans	Список своих планов
POST	/api/plans	Сохранение нового плана
GET	/api/plans/active	Активный план
GET	/api/plans/{id}	Один план
PATCH	/api/plans/{id}	Переименование плана
DELETE	/api/plans/{id}	Удаление плана с темами и занятиями
POST	/api/plans/{id}/activate	Сделать план активным
PUT	/api/plans/{id}/settings	Новые настройки расписания
GET	/api/plans/{id}/topics	Темы плана по порядку
PUT	/api/plans/{id}/topics	Новый список тем плана
GET	/api/plans/{id}/sessions	Занятия плана
PUT	/api/plans/{id}/sessions/open	Новое расписание невыполненных занятий

Метод	Адрес	Что делает
GET	/api/sessions/completed	Выполненные занятия (статистика и история)
GET	/api/sessions/{id}	Одно занятие
PUT	/api/sessions/{id}/done	Отметить занятие выполненным или снять отметку
GET	/api/custom-topics	Свои темы
POST	/api/custom-topics	Новая своя тема
GET	/api/custom-topics/{id}	Одна своя тема
PUT	/api/custom-topics/{id}	Изменение своей темы
DELETE	/api/custom-topics/{id}	Удаление своей темы
GET	/api/favorites	Избранные темы
POST	/api/favorites/{id}/toggle	Добавить в избранное или убрать
GET	/api/notes	Заметки (все или к одной теме)
POST	/api/notes	Новая заметка
GET	/api/notes/{id}	Одна заметка
PUT	/api/notes/{id}	Изменение заметки
DELETE	/api/notes/{id}	Удаление заметки
GET	/api/pomodoros	Журнал Pomodoro
POST	/api/pomodoros	Запись завершённого интервала Pomodoro

Все адреса, кроме проверки связи, каталога, регистрации и входа, требуют токен. Полный CRUD (получение, добавление, изменение, удаление) через API есть у заметок, своих тем и планов.